

Algorithm: Logistic Regression for Network Intrusion Detection**Dataset:** [KDD Cup 1999]: [Dataset](#)**Reference Notebook:** [Notebook](#)

Task	Actions Taken	Rationale	Difficulty Level (1-10)	Time Taken
Data Understanding	Loaded KDD Cup 1999 data, verified feature distributions, managed missing values, and examined class imbalance.	To learn the structure of the data before preprocessing and modeling.	3	20 min
Data Preprocessing	Encoded categorical features, numerically scaled features with StandardScaler, dimensionally reduced using PCA, and applied SMOTE for balancing classes.	Ensures data is clean, normalized, and training-ready, improving model performance. Used Sklearn's LogisticRegression with default parameters for the first evaluation.	5	30 min
Implemented Sklearn Logistic Regression	Used Sklearn's LogisticRegression with default parameters for the first evaluation.	Provides a simple baseline to compare against optimized models.	3	30 min
Hyperparameter Tuning	Used GridSearchCV with StratifiedKFold to find the optimal values of C (regularization parameter) and type of penalty (l2).	Tunes the model by selecting the best-performing hyperparameters.	6	1 hour
Model Evaluation	Metrics used: accuracy, precision, recall, F1-score, ROC curve, precision-recall curve, and confusion matrix.	Evaluates classification performance, identifying strengths and weaknesses.	5	30 min

Conclusion & Analysis	Analyzed results, examined threshold tuning effect, and provided last conclusions regarding model performance.	Helps to understand trade-offs between false positives, recall, and model generalization.	3	20 min
----------------------------------	--	---	---	--------

1. Business Understanding

Objective:

This work is focused on the detection of network intrusions from the KDD Cup 1999 dataset using a Logistic Regression model. The processes involved include data exploration, preprocessing, building of the model, and evaluation of performance.

2. Data Understanding:

First step: EDA of the KDD Cup 1999 dataset so that its structure, distribution of features, and types of network attacks can be understood. Categorical and numerical attributes of the dataset are inspected for missing values, inconsistency, and outliers. Visualization techniques help in understanding relations between features and the target attribute, providing insights into patterns that could be signatures of intrusions. Additionally, since network intrusion datasets are class-imbalanced (i.e., a higher number of normal traffic instances than attack instances), an appropriate method for handling this imbalance is considered

3. Data Preparation:

The data is preprocessed by carrying out a series of preprocessing operations.

- Missing values and inconsistencies are handled to ensure data integrity.
- Categorical columns are one-hot encoded so that the model is able to handle non-numeric attributes properly.
- Numerical columns are normalized using StandardScaler to avoid variations in feature scales from adversely impacting the model performance.
- To reduce computational complexity without sacrificing vital information, Principal Component Analysis (PCA) is employed for dimensionality reduction.
- To balance the class, Synthetic Minority Over-sampling Technique (SMOTE) is used to generate additional samples of minority attack instances.

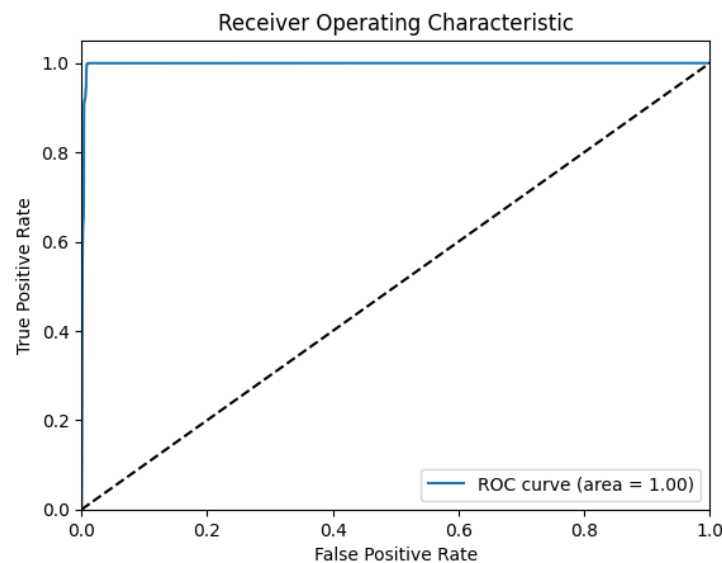
- The data is then separated into training and test sets in order to train and test models.

4. **Modeling -Baseline Implementation (custom):**

- Logistic Regression model is utilized to forecast network traffic as an attack or normal.
- Hyperparameter tuning is instrumental in enhancing the performance of the model, and GridSearchCV is employed to discover the best collection of hyperparameters like the regularization parameter (C) and the type of penalty (l2).
- The highest performing model is trained on preprocessed data in order to be able to pick up the patterns well.

5. **Evaluation:**

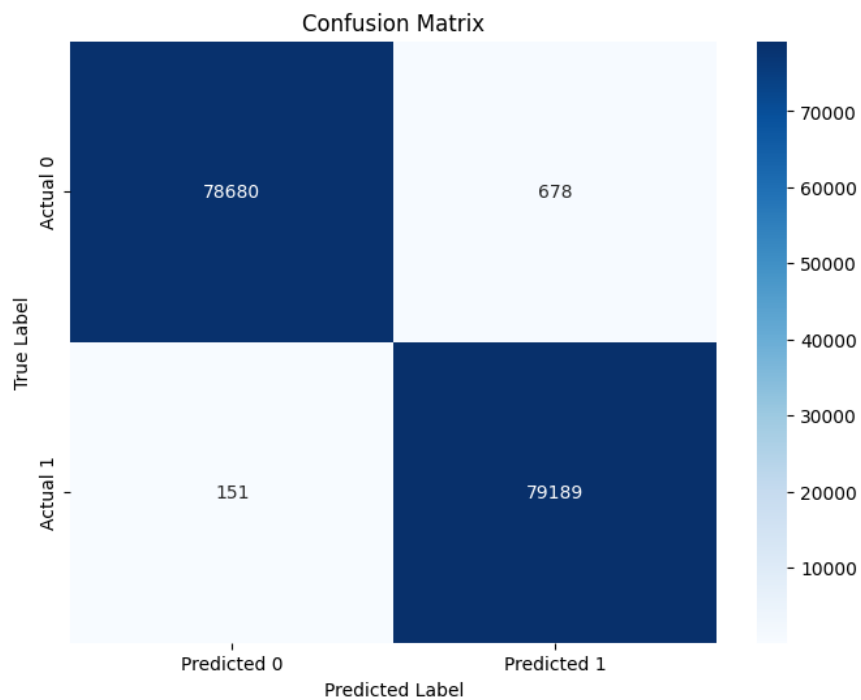
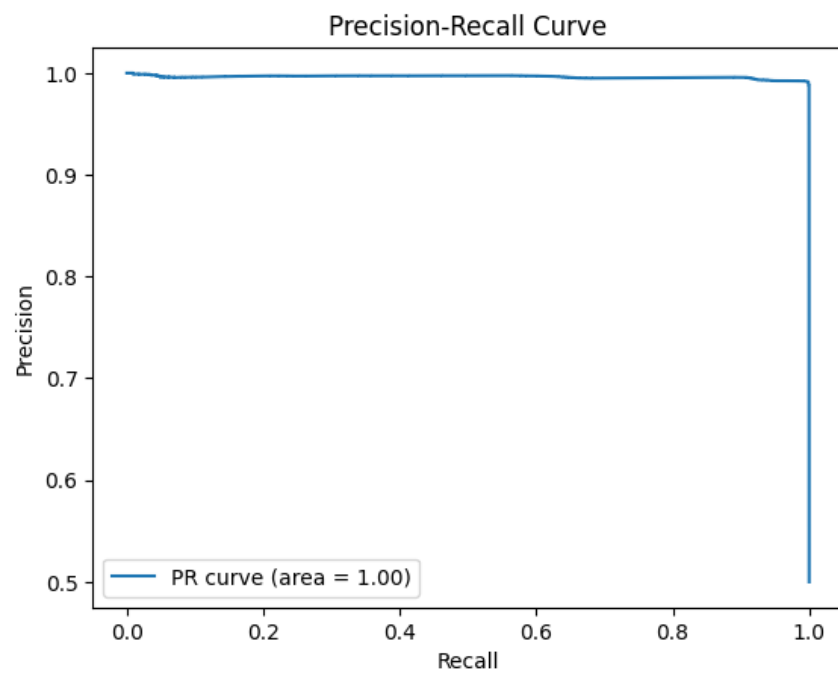
The model trained is evaluated on different performance measures like accuracy, precision, recall, F1-score, and the Area Under the ROC Curve (AUC-ROC).



A confusion matrix in order to examine classification mistakes and supply information regarding false positives and false negatives.

Receiver Operating Characteristic (ROC) curve and Precision-Recall (PR) curve are utilized to gauge the model's ability to distinguish between normal and attack traffic. Furthermore, different classification thresholds are explored in order to calibrate the sensitivity and specificity of the model relative to specific security

objectives. Finally, a formal report is created, summarizing the performance of the model, key findings, and steps taken, to aid in reproducibility for ongoing research and development.



Sensitivity (Recall): 0.9980967985883539
Specificity: 0.991456437914262
False Positive Rate: 0.008543562085738048

Observations:

- First, Logistic Regression was trained using default Sklearn parameters. GridSearchCV was used to find the optimal C (regularization strength) and penalty (l2), which improved accuracy. Stratified K-Fold cross-validation ensured robustness by testing model performance on different subsets of data.
- The best-performing model had strong accuracy, but precision-recall analysis revealed trade-offs between false positives and false negatives. The ROC curve showed a good AUC score, i.e., there was a good ability to separate normal and attack traffic. Threshold tuning analysis showed that the adjustment of classification thresholds could improve recall (which is very important for intrusion detection). The confusion matrix provided details on wrongly classified instances, which helped in refining model predictions.

6. Summary:

The Sklearn Logistic Regression model with optimized hyperparameters efficiently detected network intrusions with the optimal hyperparameters. SMOTE class balancing improved recall, ensuring attacks were detected more reliably. While Logistic Regression was sufficient as a baseline, future improvements would involve ensemble approaches (e.g., Random Forest, XGBoost) or deep learning approaches for better generalization and robustness.